

Proyecto Integrador Bootcamp

Semana 1 – Diseño del pipeline de datos

Data Raw:

clientes.csv:

```
ProyectoIntegrador > data > data_raw >  clientes.csv >  data
1 cliente_id,nombre,email,ciudad,fecha_alta
2 1,Luca Gonzalez,luca.gonzalez+1@yahoo.com,Mendoza,2023-06-02
3 2,Victoria Diaz,victoria.diaz+2@gmail.com,Tucumán,2022-03-05
4 3,Abril Peralta,abril.peralta+3@outlook.com,CABA,2021-08-04
5 4,Uma Rodriguez,uma.rodriguez+4@hotmail.com,Córdoba,2023-10-17
6 5,Ciro Gonzalez,ciro.gonzalez+5@yahoo.com,Córdoba,2022-08-25
7 6,Delfina Gimenez,delfina.gimenez+6@gmail.com,CABA,2023-02-22
8 7,Martina Diaz,martina.diaz+7@hotmail.com,Rosario,2023-02-22
9 8,Lara Molina,lara.molina+8@hotmail.com,Córdoba,2023-04-26
10 9,Delfina Nuñez,delfina.nunez+9@hotmail.com,San Luis,2020-11-11
11 10,Alma Valentina Gonzalez,almavalentina.gonzalez+10@hotmail.com,Tucumán,2023-02-04
12 11,Luciana Quiroga,luciana.quiroga+11@gmail.com,Salta,2023-06-07
13 12,Alma Fernandez,alma.fernandez+12@hotmail.com,Rosario,2022-02-27
14 13,Maximiliano Romero,maximiliano.romero+13@hotmail.com,Córdoba,2022-08-23
15 14,Catalina Gomez,catalina.gomez+14@gmail.com,Rosario,2020-01-22
16 15,Thiago Leonel Juarez,thiagoleonel.juarez+15@hotmail.com,San Luis,2021-04-17
17 16,Micaela Ojeda,micaela.ojeda+16@hotmail.com,Salta,2020-03-17
18 17,Mateo Benjamin Valdez,mateobenzamin.valdez+17@yahoo.com,Chubut,2020-06-14
19 18,Olivia Molina,olivia.molina+18@gmail.com,Santa Fe,2021-10-16
20 19,Milena Rios,milena.rios+19@outlook.com,Chubut,2022-05-16
21 20,Morena Jazmin Silva,morenajazmin.silva+20@yahoo.com,Chubut,2020-11-19
22 21,Federico Gonzalez,federico.gonzalez+21@gmail.com,Chubut,2021-03-16
23 22,Francisco Garcia,francisco.garcia+22@gmail.com,CABA,2023-12-28
24 23,Bautista Martinez,bautista.martinez+23@yahoo.com,Neuquén,2022-09-28
25 24,Santino Garcia,santino.garcia+24@yahoo.com,Santa Fe,2020-04-17
26 25,Juan Manuel Torres,juanmanuel.torres+25@gmail.com,Tucumán,2022-06-07
27 26,Thiago Diaz,thiago.diaz+26@gmail.com,Rosario,2021-02-05
28 27,Thiago Benjamin Maidana,thiagobenzamin.maidana+27@outlook.com,Neuquén,2023-12-23
29 28,Lara Cabrera,lara.cabrera+28@outlook.com,Santa Fe,2022-01-08
30 29,Joaquin Ojeda,joaquin.ojeda+29@yahoo.com,Córdoba,2023-03-03
31 30,Mateo Alvarez,mateo.alvarez+30@gmail.com,Chubut,2022-11-18
32 31,Juan Sebastian Godoy,juansebastian.godoy+31@hotmail.com,Neuquén,2021-10-21
33 32,Lautaro Benjamin Castillo,lautarobenzamin.castillo+32@hotmail.com,Neuquén,2021-05-09
34 33,Valentina Jazmin Ramos,valentinajazmin.ramos+33@yahoo.com,Neuquén,2021-08-30
35 34,Francisco Gonzalez,francisco.gonzalez+34@gmail.com,Tucumán,2021-09-15
36 35,Josefina Perez,josefina.perez+35@outlook.com,Tucumán,2023-10-24
37 36,Delfina Leiva,delfina.leiva+36@yahoo.com,Córdoba,2020-05-13
38 37,Thiago Lionel Sanchez,thiagolionel.sanchez+37@gmail.com,Córdoba,2021-09-22
39 38,Ambar Ruiz,ambar.ruiz+38@outlook.com,Rosario,2022-03-23
40 39,Uma Medina,uma.medina+39@outlook.com,Córdoba,2021-08-22
41 40,Thiago Benjamin Martinez,thiagobenzamin.martinez+40@yahoo.com,CABA,2023-11-14
```

productos.csv:

```
ProyectoIntegrador > data > data_raw > productos.csv > data
1 producto_id,nombre,categoria,precio
2 101,Oficina Item 0001,Oficina,5410.00
3 102,Smart Home Item 0002,Smart Home,14160.00
4 103,Audio Item 0003,Audio,66010.00
5 104,Oficina Item 0004,Oficina,32660.00
6 105,Gaming Item 0005,Gaming,21780.00
7 106,Accesorios Item 0006,Accesorios,3220.00
8 107,Gaming Item 0007,Gaming,99700.00
9 108,Computación Item 0008,Computación,30290.00
10 109,Smart Home Item 0009,Smart Home,102520.00
11 110,Gaming Item 0010,Gaming,175490.00
12 111,Audio Item 0011,Audio,57050.00
13 112,Redes Item 0012,Redes,13530.00
14 113,Accesorios Item 0013,Accesorios,34760.00
15 114,Redes Item 0014,Redes,7210.00
16 115,Computación Item 0015,Computación,18010.00
17 116,Redes Item 0016,Redes,43510.00
18 117,Gaming Item 0017,Gaming,161480.00
19 118,Oficina Item 0018,Oficina,110170.00
20 119,Computación Item 0019,Computación,16710.00
21 120,Smart Home Item 0020,Smart Home,109960.00
22 121,Redes Item 0021,Redes,39580.00
23 122,Oficina Item 0022,Oficina,5780.00
24 123,Accesorios Item 0023,Accesorios,18990.00
25 124,Oficina Item 0024,Oficina,75770.00
26 125,Oficina Item 0025,Oficina,16730.00
27 126,Video Item 0026,Video,135000.00
28 127,Redes Item 0027,Redes,8960.00
29 128,Redes Item 0028,Redes,10590.00
30 129,Smart Home Item 0029,Smart Home,70300.00
31 130,Oficina Item 0030,Oficina,34650.00
32 131,Audio Item 0031,Audio,30560.00
33 132,Accesorios Item 0032,Accesorios,2570.00
34 133,Computación Item 0033,Computación,37290.00
35 134,Accesorios Item 0034,Accesorios,14200.00
36 135,Gaming Item 0035,Gaming,38530.00
37 136,Gaming Item 0036,Gaming,200320.00
38 137,Computación Item 0037,Computación,36940.00
39 138,Accesorios Item 0038,Accesorios,30070.00
40 139,Redes Item 0039,Redes,10710.00
41 140,Video Item 0040,Video,50520.00
```

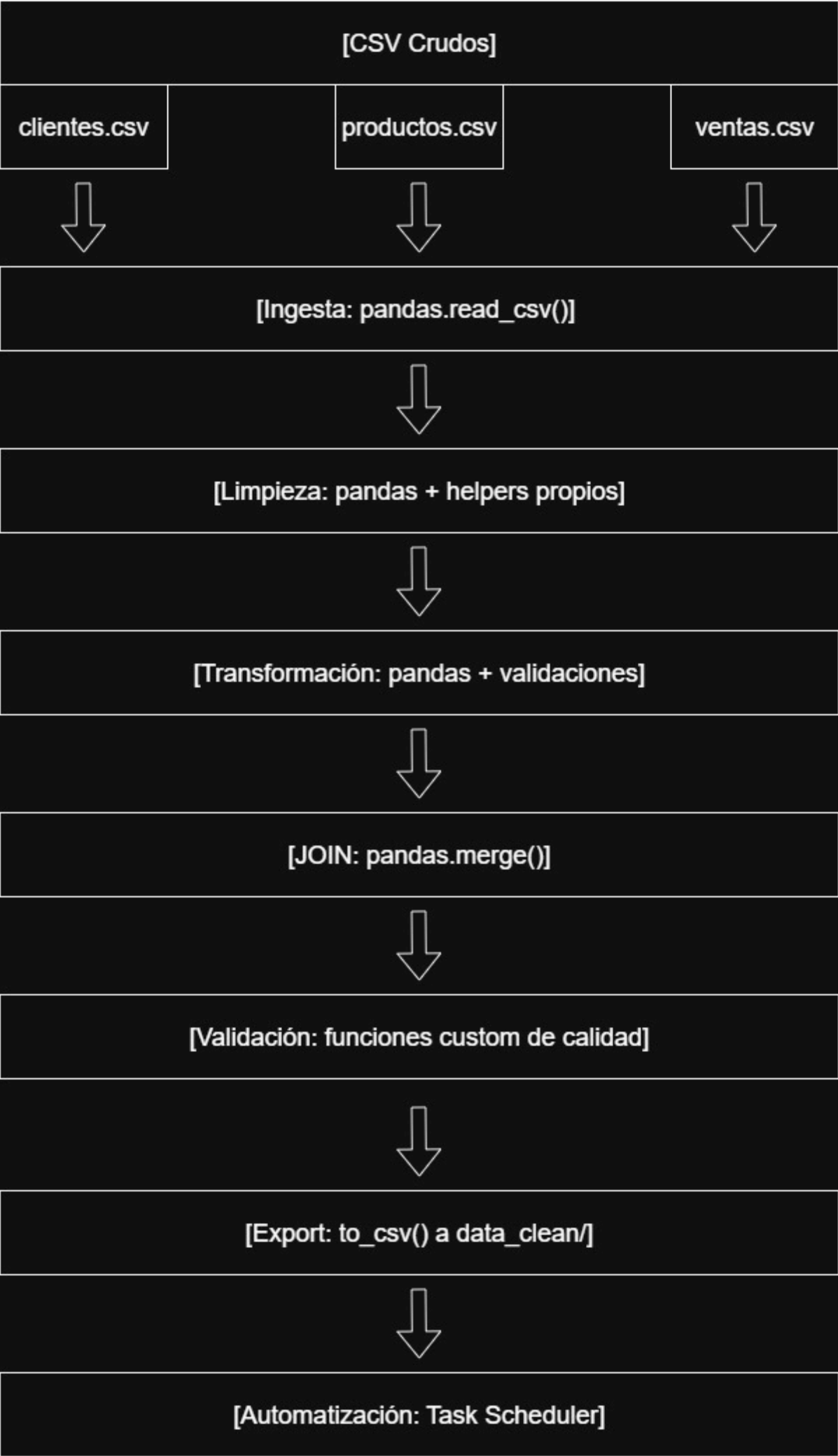
ventas.csv:

ProyectoIntegrador > data > data_raw >  ventas.csv >  data

1	<u>venta_id</u> , <u>cliente_id</u> , <u>producto_id</u> , <u>fecha_venta</u> , <u>cantidad</u> , <u>descuento</u>
2	1,3783,352,2023-10-07,1,0
3	2,1731,335,2023-05-02,4,2
4	3,7878,305,2023-01-14,1,0
5	4,80,363,2023-02-07,3,0
6	5,1303,290,2023-06-09,2,5
7	6,3510,460,2023-05-19,1,0
8	7,6246,161,2023-02-07,1,0
9	8,3249,550,2023-05-17,1,2
10	9,6549,484,2023-09-30,3,0
11	10,4418,295,2023-12-20,4,0
12	11,6525,357,2023-12-19,1,5
13	12,3793,356,2023-06-14,5,5
14	13,3251,249,2023-11-16,2,2
15	14,5158,244,2023-12-28,3,0
16	15,6722,504,2023-06-04,1,20
17	16,1413,148,2023-08-07,1,15
18	17,2494,355,2023-10-10,4,15
19	18,6168,510,2023-08-02,4,2
20	19,1622,434,2023-11-09,5,0
21	20,3403,300,2023-10-18,1,0
22	21,907,583,2023-07-31,3,2
23	22,352,505,2023-08-02,1,5
24	23,9906,394,2023-01-31,2,10
25	24,544,379,2023-02-11,1,5
26	25,4254,539,2023-11-21,3,0
27	26,4508,518,2023-07-12,1,0
28	27,7764,517,2023-06-18,1,0
29	28,7980,576,2023-07-27,3,2
30	29,2034,257,2023-10-23,2,0
31	30,4671,414,2023-09-30,1,0
32	31,2265,528,2023-01-31,3,2
33	32,5796,533,2023-06-08,2,0
34	33,128,596,2023-01-05,1,10
35	34,1096,432,2023-12-28,1,0
36	35,9503,440,2023-10-13,2,0
37	36,3017,220,2023-04-24,1,0
38	37,3488,383,2023-03-24,1,0
39	38,8428,411,2023-06-27,1,15
40	39,757,128,2023-02-02,1,2
41	40,1695,165,2023-09-06,2,2

Actividad 1

Pipeline Diagram:



Actividad 2:

Herramientas:

HERRAMIENTAS		
ETAPA	HERRAMIENTA	RAZÓN (por qué las elijo)
Ingesta	<code>pandas.read_csv()</code>	Las fuentes son CSV. Permite manejar <code>encoding</code> , <code>dtype</code> , <code>sep</code> , y <code>usecols</code> , dejando el esquema controlado desde el inicio.
Limpieza	pandas + helpers propios	Centralizo reglas reutilizables (trim, normalización de strings, casteos, formatos de fecha). Facilita testear y mantener.
Transformación	Pandas + validaciones	Aplico reglas de negocio (rangos, tipos, imputación puntual) antes del join. Trabajo local, sin frameworks pesados.
JOIN	<code>pandas.merge()</code>	Control explícito de claves y tipo de join (<code>left</code> , <code>inner</code>) sobre <code>cliente_id</code> / <code>producto_id</code> . Puedo auditar tamaños antes/después.
Validación	funciones custom de calidad	Chequeos de nulls críticos, duplicados, dominios válidos y referencialidad post-join. Simple, sin necesidad de Airflow.
Exportación	<code>DataFrame.to_csv()</code> → <code>data_clean/</code>	Entregable legible y versionable. <i>(Opcional extra: <code>to_parquet()</code>)</i>
Automatización	Task Scheduler (Windows) + script .py	Es nativo de Windows y no requiere de dependencias adicionales

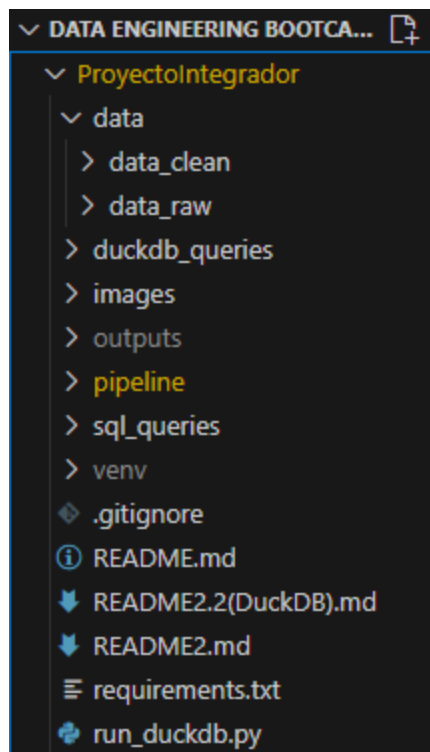
Actividad 3:

Supuestos del sistema:

- Los archivos CSV llegan una vez por día por correo (6 AM).
- Los archivos pueden venir vacíos, duplicados o con errores de formato.
- Los IDs de cliente y producto son únicos y sirven como claves de unión.
- Si un cliente tiene email inválido, igual se carga pero se marca como pendiente.
- El negocio necesita reportes diarios de ingresos y ventas por producto.
- Los archivos históricos no cambian: una vez cargados, se consideran fijos y no se reescriben.

Semana 2 – Lectura y limpieza de datos con pandas

Estructura de carpetas:



Script `.py` funcional:

```
# -*- coding: utf-8 -*-
"""
ETL de ingreso/limpieza/transformación para Semana 2 (pandas)
Estructura esperada del repo:
data/
  data_raw/ (clientes.csv, productos.csv, ventas.csv)
  data_clean/ (salidas limpias)
  outputs/ (errores, reporte_calidad, logs)
"""

from pathlib import Path
import os, re, sys, logging
import pandas as pd
import numpy as np

# =====
# Configuración de rutas
# =====
ROOT = Path(os.getenv("PROJECT_ROOT", Path.cwd())).resolve()
RAW = ROOT / "data" / "data_raw"
CLEAN = ROOT / "data" / "data_clean"
OUT = ROOT / "outputs"

if not RAW.exists():
    raise FileNotFoundError(f"No existe la carpeta de entrada: {RAW}")

CLEAN.mkdir(parents=True, exist_ok=True)
OUT.mkdir(parents=True, exist_ok=True)
```



```

# =====
# Logging básico a archivo
# =====
LOG_FILE = OUT / "pipeline.log"
logging.basicConfig(
    filename=str(LOG_FILE),
    level=logging.INFO,
    format="%asctime)s | %(levelname)s | %(message)s",
)
logger = logging.getLogger(__name__)

# =====
# Utilidades
# =====
def read_csv_robusto(path: Path, **kwargs) → pd.DataFrame:
    """Lee CSV probando UTF-8 y cae a latin-1 si es necesario."""
    try:
        return pd.read_csv(path, encoding="utf-8", **kwargs)
    except UnicodeDecodeError:
        return pd.read_csv(path, encoding="latin-1", **kwargs)

def parse_date_any(s: pd.Series) → pd.Series:
    """Parsea múltiples formatos de fecha (ISO, dd/mm/yyyy, yyyy-mm-dd, etc.)."""
    return pd.to_datetime(s, errors="coerce", dayfirst=True)

_price_re = re.compile(r"^[^d,.-]")
def parse_price(x):
    """Normaliza precios con símbolos/miles/coma; devuelve NaN si no es válido o negativo."""
    if pd.isna(x):
        return np.nan
    s = str(x).strip()
    if s.lower() in {"nan", "s/d", "sd", "-", ""}:
        return np.nan
    s = _price_re.sub("", s)
    # 1.200.000,50 → 1200000.50
    if "," in s and "." in s:
        s = s.replace(".", "").replace(",", ".")
    elif "," in s:
        s = s.replace(",", ".")
    try:
        v = float(s)
    except ValueError:
        return np.nan
    return v if v >= 0 else np.nan

def normalize_city(x):
    if pd.isna(x):
        return x
    m = str(x).strip().lower()
    mapping = {
        "caba": "CABA",
        "capital federal": "CABA",
        "cordoba": "Córdoba",
        "córdoba": "Córdoba",
        "rosario": "Rosario",
        "mendoza": "Mendoza",
        "salta": "Salta",
    }
    return mapping.get(m, str(x).strip().title())

```



```

def is_valid_email(e) → bool:
    if pd.isna(e):
        return False
    s = str(e).strip()
    return bool(re.match(r"^[^@\s]+@[^@\s]+\.[^@\s]+$", s))

# =====
# Proceso principal
# =====
def main():
    logger.info("== Inicio ETL ==")

    # ----- CLIENTES -----
    cli = read_csv_robusto(RAW / "clientes.csv", dtype=str)
    cli.columns = [c.strip().lower() for c in cli.columns]
    for c in ["nombre", "email", "ciudad", "fecha_alta"]:
        if c in cli:
            cli[c] = cli[c].astype(str).str.strip()

    cli["email_limpio"] = cli["email"].str.strip().str.lower()
    cli["email_valido"] = cli["email_limpio"].apply(is_valid_email)
    cli["ciudad"] = cli["ciudad"].apply(normalize_city)
    cli["fecha_alta_parsed"] = parse_date_any(cli["fecha_alta"])
    cli["cliente_id"] = pd.to_numeric(cli["cliente_id"], errors="coerce").astype("Int64")

    cli["_dup_full"] = cli.duplicated(keep=False)
    cli["_dup_cliente_id"] = cli.duplicated(subset=["cliente_id"], keep=False)

    cli_err = cli[
        (~cli["email_valido"]) | (cli["cliente_id"].isna()) | (cli["fecha_alta_parsed"].isna())
    ].copy()
    cli_ok = cli[~cli.index.isin(cli_err.index)].copy()

    cli_ok_final = cli_ok[
        ["cliente_id", "nombre", "email_limpio", "ciudad", "fecha_alta_parsed"]
    ].rename(columns={"email_limpio": "email", "fecha_alta_parsed": "fecha_alta"})

    # ----- PRODUCTOS -----
    prod = read_csv_robusto(RAW / "productos.csv", dtype=str)
    prod.columns = [c.strip().lower() for c in prod.columns]
    prod["producto_id"] = pd.to_numeric(prod["producto_id"], errors="coerce").astype("Int64")
    prod["nombre"] = prod["nombre"].astype(str).str.strip()
    prod["categoria"] = prod["categoria"].astype(str).str.strip().str.title()
    prod["precio_raw"] = prod["precio"]
    prod["precio"] = prod["precio"].apply(parse_price)

    prod["_dup_full"] = prod.duplicated(keep=False)
    prod["_dup_producto_id"] = prod.duplicated(subset=["producto_id"], keep=False)
    prod["precio_invalido"] = prod["precio"].isna()

    prod_err = prod[prod["producto_id"].isna() | prod["precio_invalido"] | prod["_dup_producto_id"]].copy()
    # nos quedamos con el primer precio válido por producto
    prod_sorted = prod.sort_values(["producto_id", "precio_invalido"])
    prod_ok = prod_sorted.drop_duplicates(subset=["producto_id"], keep="first")
    prod_ok_final = prod_ok[["producto_id", "nombre", "categoria", "precio"]]

    # ----- VENTAS -----
    ven = read_csv_robusto(RAW / "ventas.csv", dtype=str)
    ven.columns = [c.strip().lower() for c in ven.columns]

```



```

for c in ["venta_id", "cliente_id", "producto_id", "cantidad", "descuento"]:
    ven[c] = pd.to_numeric(ven[c], errors="coerce")
ven["fecha_venta_parsed"] = parse_date_any(ven["fecha_venta"])

ven["cantidad_valida"] = ven["cantidad"].notna() & (ven["cantidad"] > 0)
ven["desc_valido"] = ven["descuento"].notna() & (ven["descuento"].between(0, 100))
ven["_dup_venta"] = ven.duplicated(subset=["venta_id"], keep=False)

cli_ids = set(cli_ok_final["cliente_id"].dropna().astype(int))
prod_ids = set(prod_ok_final["producto_id"].dropna().astype(int))
ven["fk_cliente_ok"] = ven["cliente_id"].apply(lambda x: int(x) in cli_ids if not pd.isna(x) else False)
ven["fk_producto_ok"] = ven["producto_id"].apply(lambda x: int(x) in prod_ids if not pd.isna(x) else False)

ven_err = ven[
    ven["_dup_venta"]
    | ven["fecha_venta_parsed"].isna()
    | (~ven["cantidad_valida"])
    | (~ven["desc_valido"])
    | (~ven["fk_cliente_ok"])
    | (~ven["fk_producto_ok"])
].copy()

ven_ok = ven[~ven.index.isin(ven_err.index)].copy()
ven_ok_final = ven_ok[
    ["venta_id", "cliente_id", "producto_id", "fecha_venta_parsed", "cantidad", "descuento"]
].rename(columns={"fecha_venta_parsed": "fecha_venta"})

# ----- Export -----
cli_ok_final.to_csv(CLEAN / "clientes_clean.csv", index=False)
prod_ok_final.to_csv(CLEAN / "productos_clean.csv", index=False)
ven_ok_final.to_csv(CLEAN / "ventas_clean.csv", index=False)

cli_err.to_csv(OUT / "errores_clientes.csv", index=False)
prod_err.to_csv(OUT / "errores_productos.csv", index=False)
ven_err.to_csv(OUT / "errores_ventas.csv", index=False)

quality = pd.DataFrame(
    [
        {"tabla": "clientes", "filas_raw": len(cli), "filas_ok": len(cli_ok_final), "errores": len(cli_err)},
        {"tabla": "productos", "filas_raw": len(prod), "filas_ok": len(prod_ok_final), "errores": len(prod_err)},
        {"tabla": "ventas", "filas_raw": len(ven), "filas_ok": len(ven_ok_final), "errores": len(ven_err)},
    ]
)
quality.to_csv(OUT / "reporte_calidad.csv", index=False)

logger.info("ETL OK | data_clean/ actualizado | errores y reporte en outputs/")
print("OK - Limpios en data/data_clean, errores en outputs/, resumen en outputs/reporte_calidad.csv")

if __name__ == "__main__":
    try:
        main()
    except Exception as e:
        logger.exception("Fallo del ETL")
        print(f"ERROR: {e}", file=sys.stderr)
        sys.exit(1)

```

Output generado (**ventas_limpio.csv**):

clientes_clean.csv:

```
ProyectoIntegrador > data > data_clean > clientes_clean.csv > data
1 cliente_id,nombre,email,ciudad,fecha_alta
2 1,Luca Gonzalez,luca.gonzalez+1@yahoo.com,Mendoza,2023-02-06
3 2,Victoria Diaz,victoria.diaz+2@gmail.com,Tucumán,2022-05-03
4 3,Abril Peralta,abril.peralta+3@outlook.com,CABA,2021-04-08
5 9,Delfina Nuñez,delfina.nunez+9@hotmail.com,San Luis,2020-11-11
6 10,Alma Valentina Gonzalez,almavalentina.gonzalez+10@hotmail.com,Tucumán,2023-04-02
7 11,Luciana Quiroga,luciana.quiroga+11@gmail.com,Salta,2023-07-06
8 25,Juan Manuel Torres,juanmanuel.torres+25@gmail.com,Tucumán,2022-07-06
9 26,Thiago Diaz,thiago.diaz+26@gmail.com,Rosario,2021-05-02
10 28,Lara Cabrera,lara.cabrera+28@outlook.com,Santa Fe,2022-08-01
11 29,Joaquin Ojeda,joaquin.ojeda+29@yahoo.com,Córdoba,2023-03-03
12 32,Lautaro Benjamin Castillo,lautarobenjamin.castillo+32@hotmail.com,Neuquén,2021-09-05
13 49,Máximo Pereyra,maximo.pereyra+49@hotmail.com,Rosario,2022-06-04
14 52,Thiago Ezequiel Rojas,thiogoezequiel.rojas+52@hotmail.com,Neuquén,2021-04-09
15 56,Thiago Lionel Perez,thiagolionel.perez+56@gmail.com,Córdoba,2021-01-02
16 57,Guillermina Lopez,guillermina.lopez+57@yahoo.com,Santa Fe,2022-05-06
17 58,Ian Ezequiel Agüero,ianezequiel.agüero+58@yahoo.com,Salta,2020-10-11
18 60,Tomas Fernandez,tomas.fernandez+60@outlook.com,San Luis,2021-11-12
19 67,Enzo Leiva,enzo.leiva+67@outlook.com,Córdoba,2021-02-08
20 71,Agustin Molina,agustin.molina+71@yahoo.com,CABA,2023-11-11
21 72,Thiago Benjamin Hernandez,thiagobenjamin.hernandez+72@hotmail.com,San Luis,2021-02-11
22 81,Bautista Morales,bautista.morales+81@outlook.com,Mendoza,2020-10-09
23 83,Alma Mia Rodriguez,almamia.rodriguez+83@yahoo.com,Santa Fe,2023-06-01
24 85,Delfina Fernandez,delfina.fernandez+85@outlook.com,Tucumán,2020-08-08
25 86,Santiago Nicolas Ortiz,santiagonicolas.ortiz+86@gmail.com,Santa Fe,2021-07-10
26 94,Candela Martinez,candela.martinez+94@hotmail.com,Tucumán,2022-02-06
27 96,Ciro Garcia,ciro.garcia+96@outlook.com,Córdoba,2022-06-01
28 98,Simon Guzman,simon.guzman+98@yahoo.com,Santa Fe,2022-04-02
29 103,Pilar Diaz,pilar.diaz+103@yahoo.com,Salta,2021-02-01
30 108,Gino Toledo,gino.toledo+108@outlook.com,Rosario,2022-05-09
31 109,Lautaro Benjamin Vera,lautarobenjamin.vera+109@gmail.com,Neuquén,2022-12-01
32 115,Mateo Campos,mateo.campos+115@gmail.com,Tucumán,2023-03-07
33 116,Bautista Lopez,bautista.lopez+116@hotmail.com,San Luis,2021-04-07
34 118,Francisco Gomez,francisco.gomez+118@hotmail.com,Santa Fe,2020-04-03
35 120,Mia Rios,mia.rios+120@outlook.com,Santa Fe,2020-07-12
36 121,Ciro Nuñez,ciro.nunez+121@hotmail.com,Salta,2023-10-11
37 124,Emma Ruiz,emma.ruiz+124@yahoo.com,Chubut,2022-09-02
38 136,Augusto Garcia,augusto.garcia+136@outlook.com,Córdoba,2023-10-05
39 141,Lucio Diaz,lucio.diaz+141@hotmail.com,Rosario,2022-12-03
40 142,Juan Manuel Perez,juanmanuel.perez+142@yahoo.com,Córdoba,2023-07-02
41 145,Pedro Perez,pedro.perez+145@yahoo.com,Córdoba,2020-10-06
```

productos_clean.csv:

```
ProyectoIntegrador > data > data_clean > productos_clean.csv > data

1 producto_id,nombre,categoria,precio
2 101,Oficina Item 0001,Oficina,5410.0
3 102,Smart Home Item 0002,Smart Home,14160.0
4 103,Audio Item 0003,Audio,66010.0
5 104,Oficina Item 0004,Oficina,32660.0
6 105,Gaming Item 0005,Gaming,21780.0
7 106,Accesorios Item 0006,Accesorios,3220.0
8 107,Gaming Item 0007,Gaming,99700.0
9 108,Computación Item 0008,Computación,30290.0
10 109,Smart Home Item 0009,Smart Home,102520.0
11 110,Gaming Item 0010,Gaming,175490.0
12 111,Audio Item 0011,Audio,57050.0
13 112,Redes Item 0012,Redes,13530.0
14 113,Accesorios Item 0013,Accesorios,34760.0
15 114,Redes Item 0014,Redes,7210.0
16 115,Computación Item 0015,Computación,18010.0
17 116,Redes Item 0016,Redes,43510.0
18 117,Gaming Item 0017,Gaming,161480.0
19 118,Oficina Item 0018,Oficina,110170.0
20 119,Computación Item 0019,Computación,16710.0
21 120,Smart Home Item 0020,Smart Home,109960.0
22 121,Redes Item 0021,Redes,39580.0
23 122,Oficina Item 0022,Oficina,5780.0
24 123,Accesorios Item 0023,Accesorios,18990.0
25 124,Oficina Item 0024,Oficina,75770.0
26 125,Oficina Item 0025,Oficina,16730.0
27 126,Video Item 0026,Video,135000.0
28 127,Redes Item 0027,Redes,8960.0
29 128,Redes Item 0028,Redes,10590.0
30 129,Smart Home Item 0029,Smart Home,70300.0
31 130,Oficina Item 0030,Oficina,34650.0
32 131,Audio Item 0031,Audio,30560.0
33 132,Accesorios Item 0032,Accesorios,2570.0
34 133,Computación Item 0033,Computación,37290.0
35 134,Accesorios Item 0034,Accesorios,14200.0
36 135,Gaming Item 0035,Gaming,38530.0
37 136,Gaming Item 0036,Gaming,200320.0
38 137,Computación Item 0037,Computación,36940.0
39 138,Accesorios Item 0038,Accesorios,30070.0
40 139,Redes Item 0039,Redes,10710.0
41 140,Video Item 0040,Video,50520.0
```

ventas_clean.csv:

```
ProyectoIntegrador > data > data_clean >  ventas_clean.csv >  data
1  venta_id,cliente_id,producto_id,fecha_venta,cantidad,descuento
2  1,3783,352,2023-07-10,1,0
3  15,6722,504,2023-04-06,1,20
4  17,2494,355,2023-10-10,4,15
5  19,1622,434,2023-09-11,5,0
6  24,544,379,2023-11-02,1,5
7  32,5796,533,2023-08-06,2,0
8  47,7732,466,2023-11-05,1,0
9  49,5337,294,2023-10-04,2,5
10 51,1061,528,2023-10-07,5,0
11 57,4948,548,2023-01-12,1,5
12 64,9022,597,2023-01-03,1,0
13 88,7561,486,2023-10-10,1,5
14 89,6244,217,2023-02-05,1,0
15 97,3590,355,2023-03-09,1,5
16 118,1323,449,2023-07-09,1,0
17 124,3787,371,2023-02-05,1,10
18 129,4686,313,2023-12-05,3,2
19 134,3046,188,2023-11-12,1,10
20 141,8423,283,2023-12-08,2,0
21 144,5993,427,2023-12-12,1,0
22 147,3831,377,2023-10-06,2,0
23 157,3805,284,2023-02-04,1,0
24 160,3434,270,2023-12-11,1,0
25 169,6109,168,2023-04-11,1,5
26 170,6572,101,2023-06-02,1,0
27 186,2302,139,2023-05-10,1,0
28 198,3206,131,2023-08-03,1,10
29 205,1067,173,2023-02-10,3,5
30 213,2158,455,2023-10-12,1,0
31 226,6538,323,2023-03-01,1,2
32 237,5415,202,2023-07-10,1,0
33 243,1231,244,2023-10-11,3,0
34 245,3225,449,2023-03-11,2,5
35 249,8472,142,2023-02-07,2,15
36 251,5633,338,2023-09-07,2,5
37 252,3557,448,2023-02-05,2,0
38 259,755,228,2023-02-04,1,2
39 260,8639,196,2023-05-07,2,0
40 263,5259,462,2023-11-09,2,0
41 272,3499,136,2023-05-08,2,0
```

reporte_calidad.csv:

```
ProyectoIntegrador > outputs >  reporte_calidad.csv >  data
1  tabla,filas_raw,filas_ok,errores
2  clientes,10000,3862,6138
3  productos,500,500,0
4  ventas,200000,30536,169464
5
```

Archivo requirements:

```
ProyectoIntegrador >  requirements.txt
1  pandas==2.2.2
2  numpy==1.26.4
3  pyarrow==16.1.0
4  Faker>=24,<25
5
```

Explicación del proceso de limpieza:

Clientes

- Normalicé texto (trim y minúsculas) y estandaricé **ciudad** (CABA, Córdoba, etc.) → para tener categorías consistentes y evitar duplicados por escritura distinta.
- Validé **email** con un patrón básico y lo pasé a minúsculas → para evitar rebotes y detectar registros no contactables.
- Parseé **fecha_alta** en múltiples formatos → para poder ordenar/filtrar por fecha sin errores.
- Tipifiqué **cliente_id** a numérico y detecté **duplicados** → para garantizar unicidad de la clave.
- Filtré filas con **id nulo**, **email inválido** o **fecha inválida** → para que los joins y KPIs no se contaminen.

Productos

- Normalicé **categoría** (Title Case) y **nombre** (trim) → para clasificar correctamente.
- Convertí **precio** a `float` aceptando `$`, puntos de miles y **coma decimal**; marqué negativos/"s/d"/NaN como inválidos → para hacer métricas monetarias fiables.
- Tipifiqué **producto_id** y resolví **duplicados** por id quedándome con el primer valor válido → para evitar doble conteo del mismo producto.
- Separé filas con **precio inválido** o **id inválido/duplicado** → para no sesgar ingresos.

Ventas

- Tipifiqué campos numéricos; parseé **fecha_venta** en múltiples formatos → para cálculos temporales correctos.
- Apliqué reglas de negocio: **cantidad > 0** y **descuento 0–100** → para impedir totales inflados o descuentos imposibles.
- Bloqueeé **venta_id** duplicado y validé **FKs** contra clientes/productos limpios → para mantener integridad referencial.
- Excluí filas con fecha inválida, cantidad/desc inválidos, **FKs rotas** o duplicados → para que los KPIs reflejen operaciones reales.

Auditoría y salidas

- Exporté datasets **limpios** (`data/data_clean/*.csv`).
- Guardé **errores** detectados por tabla (`outputs/errores/*.csv`) para revisión.
- Generé un **reporte de calidad** (`outputs/reporte_calidad.csv`) con crudos/OK/errores → transparencia y trazabilidad del proceso.

En resumen: apliqué normalización de texto, parseo robusto de fechas y precios, tipado de datos, deduplicación, validaciones de rango y **FKs**, y separé todo lo inválido para auditar sin ensuciar las métricas.

Semana 3 – Transformaciones con SQL

0) Resumen

- Generé **más filas** con `pipeline/generar_raw.py` (usa Faker) y limpié con `pipeline/etl_ingesta.py` .
- Cargué en **SQL Server** las **3 tablas con todas las columnas** usando **BULK INSERT**.
- Dejé las **consultas** pedidas (sin aplicar descuento) listas para ejecutar.

1) Generación y limpieza de datos

Parámetros (editables) en `pipeline/generar_raw.py`

```
N_CLIENTES = 10_000
N_PRODUCTOS = 500
```



```
N_VENTAS    = 200_000
```

Comandos

```
# activar venv e instalar deps
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install pandas numpy Faker

# generar raw y limpiar a *_clean.csv
python .\pipeline\generar_raw.py
python .\pipeline\etl_ingesta.py

# copiar los clean a una carpeta simple para BULK
New-Item -ItemType Directory -Path "C:\Temp\week3" -Force | Out-Null
Copy-Item ".\data\data_clean\*.csv" "C:\Temp\week3\" -Force
```

Archivos usados para cargar

- `data/data_clean/productos_clean.csv` — `producto_id,nombre,categoria,precio`
- `data/data_clean/clientes_clean.csv` — `cliente_id,nombre,email,ciudad,fecha_alta`
- `data/data_clean/ventas_clean.csv` — `venta_id,cliente_id,producto_id,fecha_venta,cantidad,descuento`

2) SQL Server — scripts y orden de ejecución

Base: ProyectoBootcamp.

1. Creación + BULK: `sql_queries/InicialSQLQuery1.sql`

- Borra si existen y crea **dbo.productos**, **dbo.clientes**, **dbo.ventas** con **todas las columnas**.
- Hace `BULK INSERT` desde `C:\Temp\week3\*.csv`.
- Incluye chequeos de conteo.

2. Llaves y checks: `sql_queries/LlavesForaneasSQLQuery.sql`

- `FK_ventas_clientes` (`ventas.cliente_id` → `clientes.cliente_id`)
- `FK_ventas_productos` (`ventas.producto_id` → `productos.producto_id`)
- `CHECK cantidad > 0`
- `CHECK descuento BETWEEN 0 AND 100`

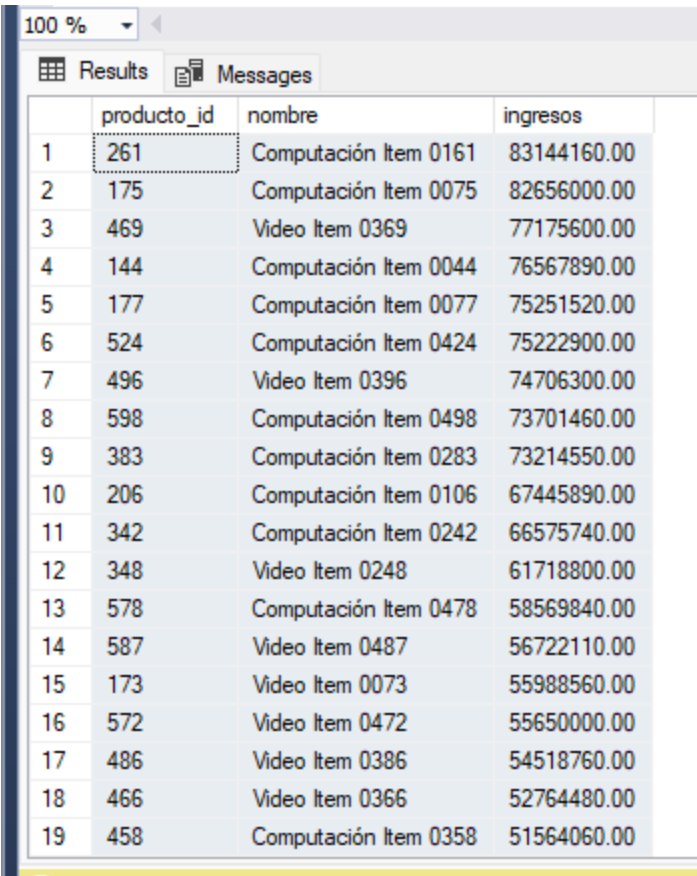
3) Consultas (sin descuento)

3.1 `sql_queries/Ej1SQLQuery.sql` — Ingresos por producto

```
/* JOIN ventas↔productos; ingresos=cantidad*precio; GROUP BY; ORDER BY desc */
USE ProyectoBootcamp;
GO

SELECT
  p.producto_id,
  p.nombre,
  CAST(SUM(v.cantidad * p.precio) AS DECIMAL(18,2)) AS ingresos
FROM dbo.ventas AS v
JOIN dbo.productos AS p
```

```
ON p.producto_id = v.producto_id
GROUP BY p.producto_id, p.nombre
ORDER BY ingresos DESC;
```



	producto_id	nombre	ingresos
1	261	Computación Item 0161	83144160.00
2	175	Computación Item 0075	82656000.00
3	469	Video Item 0369	77175600.00
4	144	Computación Item 0044	76567890.00
5	177	Computación Item 0077	75251520.00
6	524	Computación Item 0424	75222900.00
7	496	Video Item 0396	74706300.00
8	598	Computación Item 0498	73701460.00
9	383	Computación Item 0283	73214550.00
10	206	Computación Item 0106	67445890.00
11	342	Computación Item 0242	66575740.00
12	348	Video Item 0248	61718800.00
13	578	Computación Item 0478	58569840.00
14	587	Video Item 0487	56722110.00
15	173	Video Item 0073	55988560.00
16	572	Video Item 0472	55650000.00
17	486	Video Item 0386	54518760.00
18	466	Video Item 0366	52764480.00
19	458	Computación Item 0358	51564060.00

3.2 `sql_queries/Ej2SQLQuery.sql` — Productos por encima del promedio (CTEs)

```
/* CTE1: ingresos por producto; CTE2: promedio global; filtro > promedio */
USE ProyectoBootcamp;

WITH ingresos_por_producto AS (
  SELECT v.producto_id,
         SUM(CAST(v.cantidad AS DECIMAL(18,2)) * p.precio) AS ingresos
  FROM dbo.ventas v
  JOIN dbo.productos p ON p.producto_id = v.producto_id
  GROUP BY v.producto_id
),
promedio AS (
  SELECT AVG(ingresos) AS avg_ingreso
  FROM ingresos_por_producto
)
SELECT ipp.producto_id, p.nombre,
       CAST(ipp.ingresos AS DECIMAL(18,2)) AS ingresos
FROM ingresos_por_producto ipp
JOIN dbo.productos p ON p.producto_id = ipp.producto_id
CROSS JOIN promedio pr
WHERE ipp.ingresos > pr.avg_ingreso
ORDER BY ingresos DESC;
```


	producto_id	nombre	ingresos
1	261	Computación Item 0161	83144160.00
2	175	Computación Item 0075	82656000.00
3	469	Video Item 0369	77175600.00
4	144	Computación Item 0044	76567890.00
5	177	Computación Item 0077	75251520.00
6	524	Computación Item 0424	75222900.00
7	496	Video Item 0396	74706300.00
8	598	Computación Item 0498	73701460.00
9	383	Computación Item 0283	73214550.00
10	206	Computación Item 0106	67445890.00
11	342	Computación Item 0242	66575740.00
12	348	Video Item 0248	61718800.00
13	578	Computación Item 0478	58569840.00
14	587	Video Item 0487	56722110.00
15	173	Video Item 0073	55988560.00
16	572	Video Item 0472	55650000.00
17	486	Video Item 0386	54518760.00
18	466	Video Item 0366	52764480.00
19	458	Computación Item 0358	51564060.00

3.3 `sql_queries/Ej3SQLQuery.sql` — Ventas por región y participación %

```
/* CTE1: ingresos por región (clientes.ciudad como 'region');
   CTE2: total global; select final con % */
USE ProyectoBootcamp;
GO

WITH ingresos_region AS (
  SELECT c.ciudad AS region,
         SUM(CAST(v.cantidad AS DECIMAL(18,2)) * p.precio) AS ingresos
  FROM dbo.ventas v
  JOIN dbo.clientes c ON c.cliente_id = v.cliente_id
  JOIN dbo.productos p ON p.producto_id = v.producto_id
  GROUP BY c.ciudad
),
total_global AS (
  SELECT SUM(ingresos) AS total
  FROM ingresos_region
)
SELECT
  ir.region,
  CAST(ir.ingresos AS DECIMAL(18,2)) AS ingresos,
  CAST(ir.ingresos * 100.0 / tg.total AS DECIMAL(6,2)) AS porcentaje
FROM ingresos_region ir
CROSS JOIN total_global tg
ORDER BY porcentaje DESC;
```

Results		Messages	
	region	ingresos	porcentaje
1	Córdoba	581259370.00	11.05
2	CABA	559852320.00	10.64
3	Mendoza	547137810.00	10.40
4	Salta	547098970.00	10.40
5	Neuquén	532458270.00	10.12
6	Rosario	514883750.00	9.79
7	Santa Fe	509722850.00	9.69
8	Chubut	494069550.00	9.39
9	San Luis	488157920.00	9.28
10	Tucumán	485289390.00	9.23

4) Estructura relevante del repo

```
ProyectoIntegrador/  
├─ data/  
│   ├── data_raw/  
│   └─ data_clean/  
├─ pipeline/  
│   ├── generar_raw.py  
│   └─ etl_ingesta.py  
├─ sql_queries/  
│   ├── InicialSQLQuery1.sql  
│   ├── LlavesForaneasSQLQuery.sql  
│   ├── Ej1SQLQuery.sql  
│   ├── Ej2SQLQuery.sql  
│   └─ Ej3SQLQuery.sql  
└─ README2.md
```

5) Runbook rápido

1. `python pipeline/generar_raw.py`
2. `python pipeline/etl_ingesta.py`
3. `Copy-Item .\data\data_clean\*.csv C:\Temp\week3\ -Force`
4. Ejecutar **InicialSQLQuery1.sql** → crea y carga tablas
5. Ejecutar **LlavesForaneasSQLQuery.sql**
6. Ejecutar **Ej1SQLQuery.sql**, **Ej2SQLQuery.sql**, **Ej3SQLQuery.sql** para la entrega.

Semana 3 – Transformaciones con SQL (alternativa con DuckDB)

Objetivo

Correr **las mismas consultas** de la Semana 3 (**Ej1, Ej2, Ej3 — sin descuento**) pero usando **DuckDB**, que ejecuta SQL **directo sobre los CSV limpios** sin necesidad de un servidor.

Estructura usada

```
ProyectoIntegrador/  
├─ data/  
│   └─ data_clean/  
│       ├── clientes_clean.csv
```

```
|   ├── productos_clean.csv
|   └── ventas_clean.csv
└── duckdb_queries/
    ├── run_all.sql      # script SQL que crea tablas y corre Ej1/Ej2/Ej3
└── outputs/
    ├── duckdb/
    │   ├── ej1_ingresos_por_producto.csv
    │   ├── ej2_productos_sobre_promedio.csv
    │   └── ej3_region_participacion.csv
    └── local.duckdb     # base embebida opcional con las tablas
└── run_duckdb.py       # runner en Python para ejecutar run_all.sql
```

Instalación rápida

1. Activar venv e instalar dependencias (ya están en `requirements.txt`):

```
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

1. (Si no existen) crear carpetas:

```
mkdir .\duckdb_queries -Force
mkdir .\outputs\duckdb -Force
```

Cómo correr

Opción A — Runner en Python (recomendada)

Desde la raíz del proyecto:

```
python .\run_duckdb.py
```

Salida esperada:

- CSVs generados en `outputs/duckdb/` (**ej1**, **ej2**, **ej3**).
- Archivo `outputs/duckdb/local.duckdb` con las tablas materializadas (opcional).

Opción B — Solo DuckDB

```
duckdb -init .\duckdb_queries\run_all.sql
```

¿Qué hace `duckdb_queries/run_all.sql` ?

1. **Carga CSV limpios** como **tablas** DuckDB (usa `read_csv_auto` para inferir tipos).
2. Ejecuta las 3 consultas y **materializa** resultados con `CREATE TABLE ... AS SELECT`.
3. **Exporta** cada resultado a CSV (UTF-8) en `outputs/duckdb/`.

Puntos clave:

- No hay que crear esquemas a mano; DuckDB infiere tipos (con los formatos de fecha indicados).
- Las consultas son **las mismas** que en SQL Server, adaptadas a sintaxis DuckDB.



Resultados de entrega (DuckDB)

- `outputs/duckdb/ej1_ingresos_por_producto.csv`
- `outputs/duckdb/ej2_productos_sobre_promedio.csv`
- `outputs/duckdb/ej3_region_participacion.csv`



Consultas implementadas (resumen)

- Ej1 — Ingresos por producto:** `SUM(v.cantidad * p.precio)` agrupado por producto.
- Ej2 — Encima del promedio:** CTE con ingresos por producto + CTE de `AVG(ingresos)` y filtro `> promedio`.
- Ej3 — Región y % participación:** ingresos por `clientes.ciudad` y `ingresos * 100 / total`.

Las versiones completas están dentro de `duckdb_queries/run_all.sql`.



Diferencias vs SQL Server

Aspecto	SQL Server	DuckDB
Tipo	Servidor (service)	Embebido (librería/CLI)
Fuente de datos	Tablas cargadas por BULK	CSV/Parquet directo o tablas efímeras
Persistencia	DB en instancia	Archivo <code>.duckdb</code> opcional
Uso típico	ETL/OLTP/BI	Análisis local/notebooks, exploración rápida

Ambas versiones coexisten en el repo:

- SQL Server** en `sql_queries/`
- DuckDB** en `duckdb_queries/` + `run_duckdb.py` + `outputs/duckdb/`



Troubleshooting

- Acentos raros en consola:** leer CSV con `Encoding utf8` (ver arriba).
- Rutas:** en Windows, en SQL usás `C:\...`; en DuckDB (SQL) uso rutas **relativas** `data/data_clean/...` (se resuelven desde la raíz del repo).
- “No module named duckdb”:** instalar `pip install duckdb` en el **venv** activo.
- “syntax error near CREATE”:** en DuckDB, usá `CREATE TABLE ... AS WITH ... SELECT` (ya aplicado en `run_all.sql`).



Reproducible de punta a punta (solo DuckDB)

```
# 1) Asegurar data limpia (ya creada por tu ETL en data/data_clean/)
# 2) Instalar
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt

# 3) Ejecutar
python .\run_duckdb.py

# 4) Revisar salidas
Get-ChildItem .\outputs\duckdb\
```



Semana 4 – Modelado de datos (Esquema Estrella) - DuckDB

1 Archivo `.sql` con creación de las 3 tablas

```
settings.json  requirements.txt  semana4_modelo_estrella.sql U X  README2.2(DuckDB).md  F
ProyectoIntegrador > sql_queries > semana4_modelo_estrella.sql > ...
1  -- =====
2  -- ■ SEMANA 4 - MODELO DE DATOS ESTRELLA
3  -- =====
4
5  -- ♦ Crear tabla de dimensión CLIENTES
6  CREATE OR REPLACE TABLE dim_clientes AS
7  SELECT DISTINCT
8      cliente_id,
9      nombre AS nombre_cliente,
10     email,
11     ciudad,
12     fecha_alta
13 FROM 'data/data_clean/clientes_clean.csv';
14
15 -- ♦ Crear tabla de dimensión PRODUCTOS
16 CREATE OR REPLACE TABLE dim_productos AS
17 SELECT DISTINCT
18     producto_id,
19     nombre AS nombre_producto,
20     categoria,
21     precio
22 FROM 'data/data_clean/productos_clean.csv';
23
24 -- ♦ Crear tabla de dimensión FECHAS
25 CREATE OR REPLACE TABLE dim_fechas AS
26 SELECT DISTINCT
27     fecha_venta,
28     EXTRACT(year FROM fecha_venta) AS anio,
29     EXTRACT(month FROM fecha_venta) AS mes,
30     EXTRACT(day FROM fecha_venta) AS dia
31 FROM 'data/data_clean/ventas_clean.csv';
32
33 -- ♦ Crear tabla de hechos FACT_VENTAS
34 CREATE OR REPLACE TABLE fact_ventas AS
35 SELECT
36     v.venta_id,
37     v.cliente_id,
38     v.producto_id,
39     v.fecha_venta,
40     v.cantidad,
41     v.descuento,
42     p.precio,
43     ROUND(v.cantidad * p.precio * (1 - v.descuento / 100.0), 2) AS ingreso_total
44 FROM 'data/data_clean/ventas_clean.csv' v
45 JOIN dim_productos p ON v.producto_id = p.producto_id;
46
```

```
47 -- =====
48 -- ✅ EXPORTS (archivos de evidencia)
49 -- =====
50 COPY dim_clientes TO 'outputs/duckdb/dim_clientes.csv' (HEADER, DELIMITER ',');
51 COPY dim_productos TO 'outputs/duckdb/dim_productos.csv' (HEADER, DELIMITER ',');
52 COPY fact_ventas TO 'outputs/duckdb/fact_ventas.csv' (HEADER, DELIMITER ',');
53
54 -- =====
55 -- 🔍 VALIDACIONES (opcional)
56 -- =====
57 -- ¿todas las ventas tienen cliente en la dimensión?
58 SELECT COUNT(*) AS clientes_faltantes
59 FROM fact_ventas
60 WHERE cliente_id NOT IN (SELECT cliente_id FROM dim_clientes);
61
62 -- ¿todas las ventas tienen producto en la dimensión?
63 SELECT COUNT(*) AS productos_faltantes
64 FROM fact_ventas
65 WHERE producto_id NOT IN (SELECT producto_id FROM dim_productos);
66
67 -- Conteos rápidos
68 SELECT
69   (SELECT COUNT(*) FROM dim_clientes) AS filas_dim_clientes,
70   (SELECT COUNT(*) FROM dim_productos) AS filas_dim_productos,
71   (SELECT COUNT(*) FROM fact_ventas) AS filas_fact_ventas;
72
```

2 Captura de las 3 tablas resultantes

dim_clientes.csv:

```
ProyectoIntegrador > outputs > duckdb > 📄 dim_clientes.csv > 📄 data
1 cliente_id,nombre_cliente,email,ciudad,fecha_alta
2 86,Santiago Nicolas Ortiz,santiagonicolas.ortiz+86@gmail.com,Santa Fe,2021-07-10
3 124,Emma Ruiz,emma.ruiz+124@yahoo.com,Chubut,2022-09-02
4 141,Lucio Diaz,lucio.diaz+141@hotmail.com,Rosario,2022-12-03
5 156,Tiziano Valentin Diaz,tizianovalentin.diaz+156@gmail.com,San Luis,2022-09-12
6 182,Sofía Gutierrez,sofia.gutierrez+182@hotmail.com,Tucumán,2021-12-06
7 223,Uma Cabrera,uma.cabrera+223@hotmail.com,Tucumán,2023-11-06
8 276,Helena Morales,helena.morales+276@yahoo.com,Neuquén,2023-09-04
9 285,Juan Manuel Rivero,juanmanuel.rivero+285@gmail.com,Rosario,2020-08-01
10 309,Thiago Valentin Romero,thiagovalentin.romero+309@hotmail.com,CABA,2022-01-09
11 337,Juan Sebastian Sanchez,juansebastian.sanchez+337@yahoo.com,Chubut,2020-04-07
12 341,Antonella Fernandez,antonella.fernandez+341@hotmail.com,Mendoza,2021-03-11
13 410,Maria Emilia Vega,mariaemilia.vega+410@hotmail.com,Mendoza,2023-08-08
14 411,Nina Alvarez,nina.alvarez+411@yahoo.com,Salta,2020-08-03
15 416,Francisca Benitez,francisca.benitez+416@outlook.com,Rosario,2022-03-04
16 425,Lucio General,lucio_general+425@yahoo.com,Córdoba,2022-01-02
```

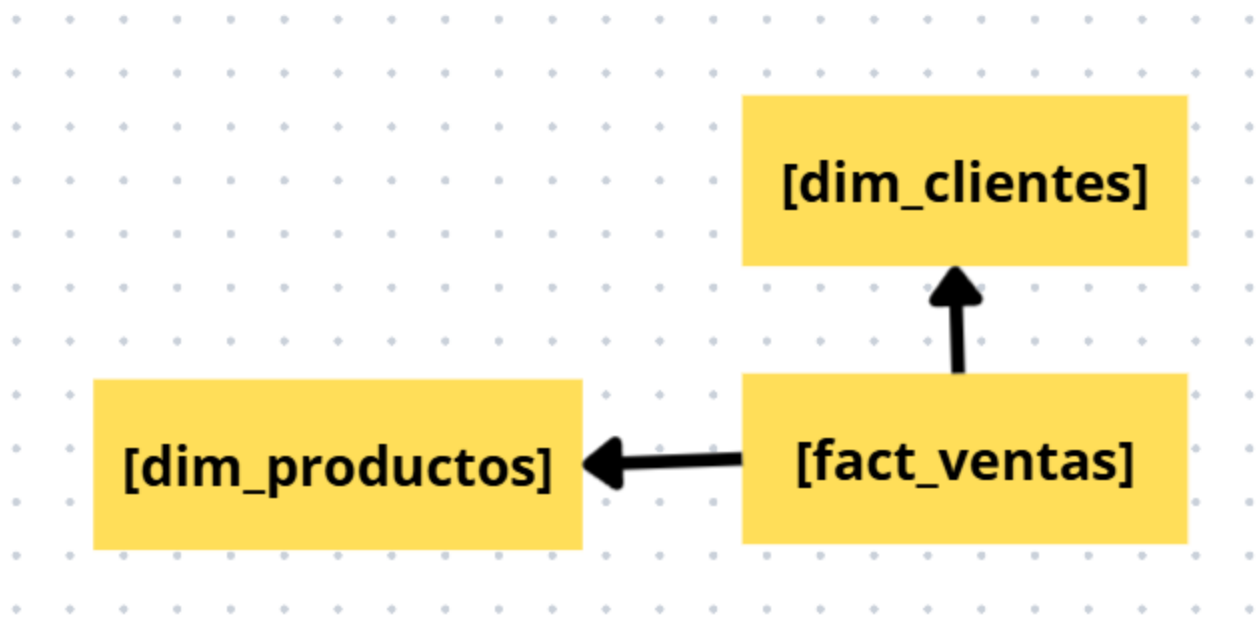
dim_productos.csv:


```
ProyectoIntegrador > outputs > duckdb >  dim_productos.csv >  data
1 producto_id,nombre_producto,categoria,precio
2 101,Oficina Item 0001,Oficina,5410.0
3 118,Oficina Item 0018,Oficina,110170.0
4 123,Accesorios Item 0023,Accesorios,18990.0
5 129,Smart Home Item 0029,Smart Home,70300.0
6 134,Accesorios Item 0034,Accesorios,14200.0
7 150,Computación Item 0050,Computación,146880.0
8 157,Computación Item 0057,Computación,23320.0
9 168,Smart Home Item 0068,Smart Home,13890.0
10 176,Accesorios Item 0076,Accesorios,15590.0
11 195,Oficina Item 0095,Oficina,6060.0
12 206,Computación Item 0106,Computación,531070.0
13 218,Oficina Item 0118,Oficina,33130.0
14 231,Audio Item 0131,Audio,139620.0
15 243,Accesorios Item 0143,Accesorios,14340.0
16 247,Computación Item 0147,Computación,390690.0
17 258,Gaming Item 0158,Gaming,262080.0
18 260,Oficina Item 0160,Oficina,15390.0
19 283,Redes Item 0183,Redes,23890.0
20 293,Smart Home Item 0193,Smart Home,41920.0
21 352,Video Item 0252,Video,21040.0
22 377,Smart Home Item 0277,Smart Home,118320.0
23 414,Audio Item 0314,Audio,8290.0
24 423,Video Item 0323,Video,15220.0
```

fact_ventas.csv:

```
ProyectoIntegrador > outputs > duckdb >  fact_ventas.csv >  data
1 venta_id,cliente_id,producto_id,fecha_venta,cantidad,descuento,precio,ingreso_total
2 1,3783,352,2023-07-10,1,0,21040.0,21040.0
3 15,6722,504,2023-04-06,1,20,345590.0,276472.0
4 17,2494,355,2023-10-10,4,15,76500.0,260100.0
5 19,1622,434,2023-09-11,5,0,153130.0,765650.0
6 24,544,379,2023-11-02,1,5,54030.0,51328.5
7 32,5796,533,2023-08-06,2,0,50610.0,101220.0
8 47,7732,466,2023-11-05,1,0,425520.0,425520.0
9 49,5337,294,2023-10-04,2,5,5970.0,11343.0
10 51,1061,528,2023-10-07,5,0,198200.0,991000.0
11 57,4948,548,2023-01-12,1,5,217900.0,207005.0
12 64,9022,597,2023-01-03,1,0,70740.0,70740.0
13 88,7561,486,2023-10-10,1,5,368370.0,349951.5
14 89,6244,217,2023-02-05,1,0,74820.0,74820.0
15 97,3590,355,2023-03-09,1,5,76500.0,72675.0
16 118,1323,449,2023-07-09,1,0,13380.0,13380.0
17 124,3787,371,2023-02-05,1,10,157380.0,141642.0
18 129,4686,313,2023-12-05,3,2,2440.0,7173.6
19 134,3046,188,2023-11-12,1,10,8810.0,7929.0
20 141,8423,283,2023-12-08,2,0,23890.0,47780.0
21 144,5993,427,2023-12-12,1,0,41200.0,41200.0
22 147,3831,377,2023-10-06,2,0,118320.0,236640.0
23 157,3805,284,2023-02-04,1,0,17010.0,17010.0
24 160,3434,270,2023-12-11,1,0,233500.0,233500.0
```

 Diagrama visual (simplificado):



4 Breve descripción:

¿Qué tabla contiene los hechos?

La tabla que contiene los hechos es `fact_ventas`.

Cada fila representa una venta individual, incluyendo el identificador del cliente, del producto, la fecha de venta, la cantidad, el descuento aplicado, el precio del producto y el **ingreso total calculado**.

¿Qué atributos hay en cada dimensión?

- `dim_clientes`
 - cliente_id
 - nombre_cliente
 - email
 - ciudad
 - fecha_alta
- `dim_productos`
 - producto_id
 - nombre_producto
 - categoría
 - precio

¿Qué métricas podría sacar de este modelo?

- Ingresos totales por producto, categoría o cliente.
- Ventas por ciudad o región.
- Descuento promedio aplicado por cliente o producto.
- Productos más vendidos o más rentables.
- Promedio de ingresos por transacción o por cliente.

Semana 5 – Automatización del Pipeline

Objetivo

Automatizar la ejecución completa del pipeline de datos para que se ejecute **sin intervención manual**, de forma **reproducible y programada**, incluyendo:

- Ingesta y limpieza
- Transformaciones SQL
- Modelado analítico
- Generación de outputs
- Registro de logs

Enfoque adoptado

En lugar de mantener un loop activo con `schedule` dentro de Python, se utilizó **Windows Task Scheduler**, que cumple el mismo rol que `cron` en Linux y es una solución realista en entornos Windows.

El pipeline se ejecuta **una vez por día** y deja evidencia verificable mediante archivos de salida y logs.

Herramientas utilizadas

Componente	Herramienta	Motivo
Orquestación	<code>run_pipeline.py</code>	Script central que ejecuta todas las etapas
SQL Engine	DuckDB	Permite ejecutar SQL directamente sobre CSV
Entorno	<code>venv</code>	Aislamiento de dependencias
Automatización	Windows Task Scheduler	Ejecución programada diaria
Logging	<code>logging</code> (Python)	Trazabilidad del pipeline

Script principal de automatización

Archivo: `run_pipeline.py`

Responsabilidad del script:

1. Ejecutar consultas SQL analíticas (`duckdb_queries/run_all.sql`)
2. Ejecutar el modelo estrella (`sql_queries/semana4_modelo_estrella.sql`)
3. Generar outputs finales (`outputs/duckdb/`)
4. Registrar la ejecución en un archivo de log

Salida esperada:

- `outputs/run_pipeline.log`
- CSVs actualizados en `outputs/duckdb/`
- Base local `outputs/duckdb/local.duckdb`

Programación de la tarea

La ejecución fue automatizada usando **Windows Task Scheduler** con la siguiente configuración:

- **Frecuencia:** diaria
- **Horario:** 06:00 AM
- **Program/script:**

```
C:\...\ProyectoIntegrador\venv\Scripts\python.exe
```

- **Add arguments:**

```
C:\...\ProyectoIntegrador\run_pipeline.py
```

- **Start in:**

```
C:\...\ProyectoIntegrador
```

Esto garantiza que:

- Se use el Python correcto del entorno virtual
- Las rutas relativas funcionen correctamente
- El pipeline pueda ejecutarse sin sesión abierta

Validación de ejecución

La correcta ejecución del pipeline se verifica mediante:

1. Archivo de log

```
outputs/run_pipeline.log
```

Ejemplo de salida:

```
INFO | EjecutandoSQL: run_all.sql
INFO | OKSQL
INFO | Ejecutando modelo estrella
INFO | == Pipeline completo OK ==
```

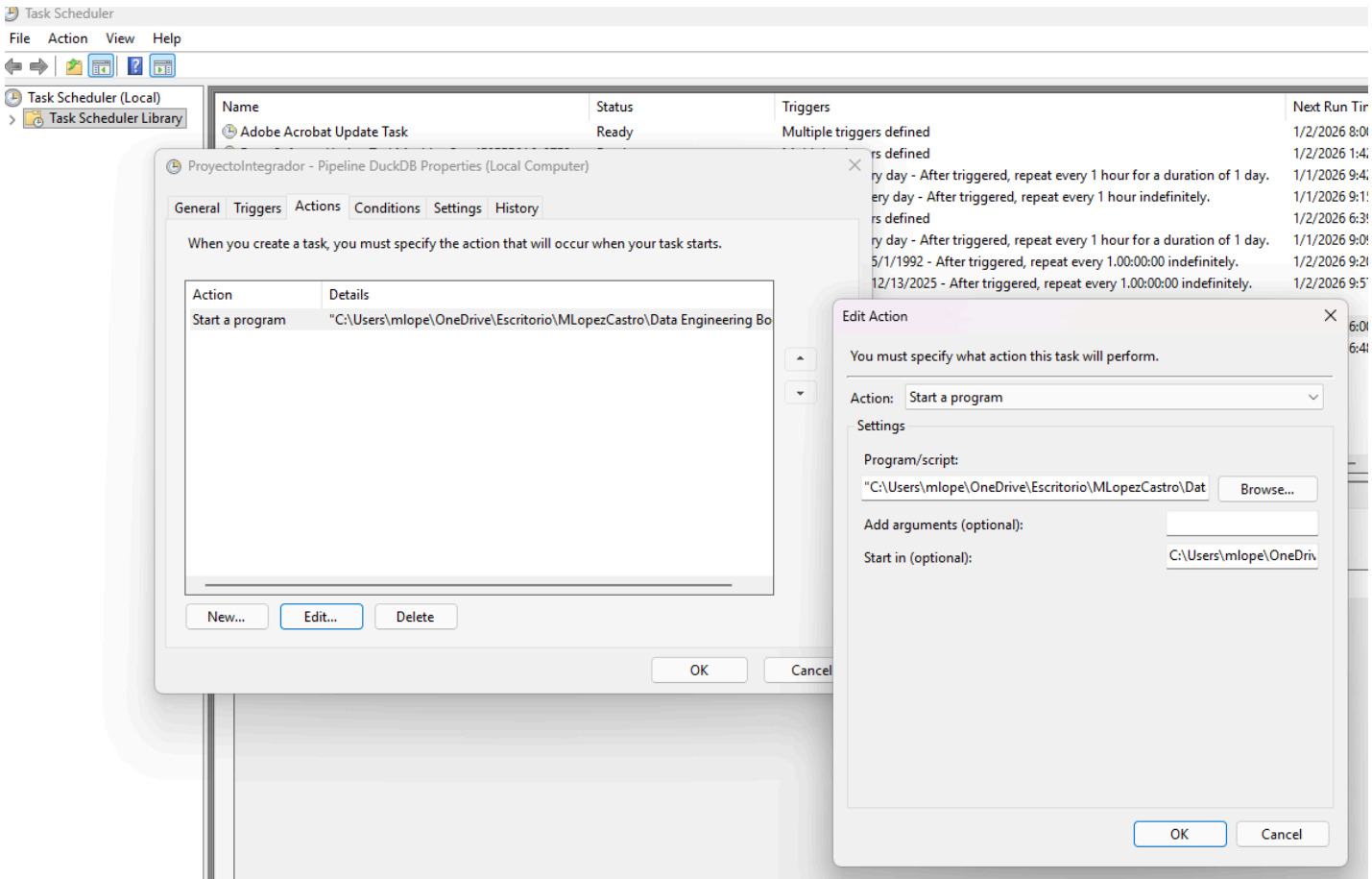
2. Archivos generados

- `outputs/duckdb/fact_ventas.csv`
- `outputs/duckdb/dim_clientes.csv`
- `outputs/duckdb/ej1_ingresos_por_producto.csv`
- etc.

3. Timestamps actualizados

Los archivos se regeneran con la hora de ejecución programada.

Task Scheduler – pestaña Actions



Task Scheduler – History

General Triggers Actions Conditions Settings History					
Number of events: 117					
Level	Date and Time	Event ID	Task Category	Operational Code	Correlation Id
Information	1/1/2026 9:11:00 PM	102	Task completed	(2)	1449a5f0-08...
Information	1/1/2026 9:11:00 PM	201	Action completed	(2)	1449a5f0-08...
Information	1/1/2026 9:10:59 PM	110	Task triggered by user	Info	1449a5f0-08...
Information	1/1/2026 9:10:59 PM	200	Action started	(1)	1449a5f0-08...
Information	1/1/2026 9:10:59 PM	100	Task Started	(1)	1449a5f0-08...
Information	1/1/2026 9:10:59 PM	129	Created Task Process	Info	

Archivo `outputs/run_pipeline.log`

```
34 2026-01-01 21:06:49,648 INFO OK SQL: C:\Users\mllope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador\duckdb_queries\run_all.sql
35 2026-01-01 21:06:49,872 INFO Ejecutando SQL: C:\Users\mllope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador\sql_queries\semana4_modelo_estrella.sql
36 2026-01-01 21:06:49,872 INFO OK SQL: C:\Users\mllope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador\sql_queries\semana4_modelo_estrella.sql
37 2026-01-01 21:10:59,576 INFO == Pipeline completo OK ==
38 2026-01-01 21:10:59,576 INFO == Inicio pipeline completo ==
39 2026-01-01 21:10:59,576 INFO CWD=C:\Users\mllope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador
40 2026-01-01 21:10:59,576 INFO DB=C:\Users\mllope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador\outputs\duckdb\local.duckdb
41 2026-01-01 21:10:59,607 INFO OUT_DIR=C:\Users\mllope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador\outputs\duckdb
42 2026-01-01 21:10:59,754 INFO Ejecutando SQL: C:\Users\mllope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador\duckdb_queries\run_all.sql
43 2026-01-01 21:10:59,755 INFO OK SQL: C:\Users\mllope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador\duckdb_queries\run_all.sql
44 2026-01-01 21:10:59,977 INFO Ejecutando SQL: C:\Users\mllope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador\sql_queries\semana4_modelo_estrella.sql
45 2026-01-01 21:10:59,978 INFO OK SQL: C:\Users\mllope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador\sql_queries\semana4_modelo_estrella.sql
46 == Pipeline completo OK ==
```

Carpeta `outputs/duckdb/`

Name	Status	Date modified
dim_clientes.csv	✓	1/1/2026 9:10 PM
dim_fechas.csv	✓	1/1/2026 9:10 PM
dim_productos.csv	✓	1/1/2026 9:10 PM
ej1_ingresos_por_producto.csv	✓	1/1/2026 9:10 PM
ej2_productos_sobre_promedio.csv	✓	1/1/2026 9:10 PM
ej3_region_participacion.csv	✓	1/1/2026 9:10 PM
fact_ventas.csv	✓	1/1/2026 9:10 PM
local.duckdb	✓	1/1/2026 9:11 PM

✓ Resultado

El pipeline quedó **completamente automatizado**, ejecutándose de forma diaria, con:

- Logs persistentes
- Outputs reproducibles
- Sin necesidad de ejecución manual

Esto replica el comportamiento esperado de un pipeline de datos en un entorno productivo simple.

En entornos Linux, esta automatización podría implementarse mediante Cron.

En este proyecto se utilizó **Windows Task Scheduler**, que cumple el mismo rol y es consistente con el sistema operativo utilizado.

Semana 6 – Validación de calidad de datos

Objetivo

Incorporar una **etapa de validación automática de calidad de datos** dentro del pipeline, con el objetivo de detectar errores comunes antes de considerar el proceso como exitoso.

Las validaciones se ejecutan como un **quality gate**:

si se detectan errores críticos, el pipeline se detiene y deja evidencia en logs y archivos de salida.

¿Por qué esto importa?

Porque un pipeline que transforma datos sin validar puede propagar errores hacia modelos analíticos, dashboards o reportes finales.

Esta etapa permite:

- Asegurar la **confiabilidad de los outputs**
- Detectar problemas de datos de forma temprana
- Tener trazabilidad de errores mediante logs y reportes
- Simular un escenario real de ingeniería de datos, donde la calidad es parte del flujo y no un paso manual

Herramientas utilizadas

- **Python**
- **pandas** para análisis y validaciones de datos
- **logging** para registro de resultados
- Manejo de errores con `raise Exception` para cortar la ejecución si algo falla

Implementación: `validaciones.py`

Se desarrolló un script específico de validación ubicado dentro del módulo del pipeline:

```
pipeline/validaciones.py
```

Este script se ejecuta **después de las transformaciones y el modelado**, y valida los datos ya procesados.

Validaciones implementadas

- Detección de **valores nulos** en columnas críticas
- Validación de **rangos válidos** (por ejemplo, cantidades mayores a cero)
- Chequeo de **descuentos fuera de rango**
- Verificación de **consistencia general de los datos**

- Generación de archivos de error cuando se detectan inconsistencias

Si alguna validación falla:

- Se generan archivos `errores_*.csv`
- Se escribe el detalle en un log de validaciones
- Se lanza una excepción que **interrumpe el pipeline**

Si todas las validaciones pasan:

- El proceso continúa
- Se registra el éxito en los logs

Outputs generados

Durante esta etapa se generan los siguientes archivos:

- `outputs/validaciones.log` → log detallado de la ejecución de validaciones
- `outputs/reporte_calidad.csv` → resumen del estado de calidad
- `outputs/errores_clientes.csv`
- `outputs/errores_productos.csv`
- `outputs/errores_ventas.csv`

Estos archivos permiten auditar rápidamente qué tipo de problemas se detectaron y en qué etapa.

Integración con el pipeline (Semana 5)

Las validaciones fueron integradas al pipeline principal mediante el script `run_pipeline.py`.

El flujo completo es:

1. Ejecución de transformaciones SQL en DuckDB
2. Ejecución del modelo estrella
3. **Ejecución de `validaciones.py` como quality gate**
4. Finalización del pipeline solo si las validaciones son exitosas

Si las validaciones fallan, el pipeline se detiene automáticamente y queda registrado en los logs.

Ejecución y pruebas

Las validaciones pueden ejecutarse de forma independiente:

```
python pipeline/validaciones.py
```

O como parte del pipeline completo:

```
python run_pipeline.py
```

En ambos casos, los resultados quedan registrados en los logs y archivos de salida.

Entregables de la semana

1. Script `pipeline/validaciones.py` funcional
2. Logs de validación (`outputs/validaciones.log`)
3. Archivos de error y reporte de calidad
4. Evidencia de ejecución exitosa integrada al pipeline principal

```

# pipeline/validaciones.py
from pathlib import Path
import logging
import pandas as pd
from datetime import datetime

BASE = Path(__file__).resolve().parents[1]    # root del proyecto
CLEAN = BASE / "data" / "data_clean"
OUT = BASE / "outputs"
OUT.mkdir(parents=True, exist_ok=True)

LOG_FILE = OUT / "validaciones.log"
logging.basicConfig(
    filename=str(LOG_FILE),
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s",
)
logger = logging.getLogger("validaciones")

def _req_file(path: Path) → None:
    if not path.exists():
        raise FileNotFoundError(f"No existe: {path}")

def _warn(cond: bool, msg: str) → int:
    """Suma 1 error si cond=True y loguea warning."""
    if cond:
        logger.warning(msg)
        return 1
    return 0

def _error_and_raise(total_errores: int) → None:
    logger.error(f"❌ Validaciones FALLIDAS. Errores encontrados: {total_errores}.")
    raise Exception(f"Validaciones FALLIDAS ({total_errores}). Revisar: {LOG_FILE}")

def main() → None:
    logger.info("== Inicio validaciones (Semana 6) ==")

    # ----- Archivos requeridos -----
    f_cli = CLEAN / "clientes_clean.csv"
    f_pro = CLEAN / "productos_clean.csv"
    f_ven = CLEAN / "ventas_clean.csv"
    _req_file(f_cli)
    _req_file(f_pro)
    _req_file(f_ven)

    # ----- Carga -----
    clientes = pd.read_csv(f_cli)
    productos = pd.read_csv(f_pro)
    ventas = pd.read_csv(f_ven)

    errores = 0

    # =====

```

```

# CLIENTES
# =====
expected_cli = {"cliente_id", "nombre", "email", "ciudad", "fecha_alta"}
errores += _warn(set(clientes.columns) != expected_cli,
                  f"Cientes: columnas inesperadas. Esperado={expected_cli}, Actual={set(clientes.columns)}")

errores += _warn(clientes["cliente_id"].isna().any(),
                  "Clientes: cliente_id contiene nulos.")
errores += _warn(clientes["cliente_id"].duplicated().any(),
                  "Clientes: cliente_id contiene duplicados.")

errores += _warn(clientes["email"].isna().any(),
                  "Clientes: email contiene nulos.")
errores += _warn((clientes["email"].astype(str).str.strip() == "").any(),
                  "Clientes: email contiene strings vacíos.")

# fecha_alta parseable + no futura (opcional)
cli_fecha = pd.to_datetime(clientes["fecha_alta"], errors="coerce", dayfirst=True)
errores += _warn(cli_fecha.isna().any(),
                  "Clientes: fecha_alta contiene valores no parseables.")
hoy = pd.Timestamp(datetime.today().date())
errores += _warn((cli_fecha > hoy).any(),
                  "Clientes: fecha_alta contiene fechas futuras.")

# =====
# PRODUCTOS
# =====
expected_pro = {"producto_id", "nombre", "categoria", "precio"}
errores += _warn(set(productos.columns) != expected_pro,
                  f"Productos: columnas inesperadas. Esperado={expected_pro}, Actual={set(productos.column
s)}")

errores += _warn(productos["producto_id"].isna().any(),
                  "Productos: producto_id contiene nulos.")
errores += _warn(productos["producto_id"].duplicated().any(),
                  "Productos: producto_id contiene duplicados.")

# precio numérico y > 0
precio_num = pd.to_numeric(productos["precio"], errors="coerce")
errores += _warn(precio_num.isna().any(),
                  "Productos: precio tiene valores no numéricos.")
errores += _warn((precio_num <= 0).any(),
                  "Productos: hay precios <= 0 (regla: precio debe ser > 0).")

# =====
# VENTAS
# =====
expected_ven = {"venta_id", "cliente_id", "producto_id", "fecha_venta", "cantidad", "descuento"}
errores += _warn(set(ventas.columns) != expected_ven,
                  f"Ventas: columnas inesperadas. Esperado={expected_ven}, Actual={set(ventas.columns)}")

# PK duplicada
errores += _warn(ventas["venta_id"].isna().any(),
                  "Ventas: venta_id contiene nulos.")
errores += _warn(ventas["venta_id"].duplicated().any(),
                  "Ventas: venta_id contiene duplicados.")

# fecha_venta válida + no futura
ven_fecha = pd.to_datetime(ventas["fecha_venta"], errors="coerce", dayfirst=True)
errores += _warn(ven_fecha.isna().any(),

```

```

        "Ventas: fecha_venta contiene valores no parseables.")
errores += _warn((ven_fecha > hoy).any(),
        "Ventas: fecha_venta contiene fechas futuras.")

# cantidad > 0
cant = pd.to_numeric(ventas["cantidad"], errors="coerce")
errores += _warn(cant.isna().any(),
        "Ventas: cantidad tiene valores no numéricos.")
errores += _warn((cant <= 0).any(),
        "Ventas: hay cantidad <= 0.")

# descuento 0-100
desc = pd.to_numeric(ventas["descuento"], errors="coerce")
errores += _warn(desc.isna().any(),
        "Ventas: descuento tiene valores no numéricos.")
errores += _warn((~desc.between(0, 100)).any(),
        "Ventas: hay descuentos fuera de rango (0-100).")

# Integridad referencial (FKs)
cli_ids = set(pd.to_numeric(clientes["cliente_id"], errors="coerce").dropna().astype(int))
pro_ids = set(pd.to_numeric(productos["producto_id"], errors="coerce").dropna().astype(int))

ven_cli = pd.to_numeric(ventas["cliente_id"], errors="coerce")
ven_pro = pd.to_numeric(ventas["producto_id"], errors="coerce")

errores += _warn(ven_cli.isna().any(),
        "Ventas: cliente_id tiene valores no numéricos o nulos.")
errores += _warn(ven_pro.isna().any(),
        "Ventas: producto_id tiene valores no numéricos o nulos.")

fk_cli_bad = ven_cli.dropna().astype(int).apply(lambda x: x not in cli_ids)
fk_pro_bad = ven_pro.dropna().astype(int).apply(lambda x: x not in pro_ids)

errores += _warn(fk_cli_bad.any(),
        f"Ventas: {int(fk_cli_bad.sum())} registros con cliente_id sin match en dim_clientes.")
errores += _warn(fk_pro_bad.any(),
        f"Ventas: {int(fk_pro_bad.sum())} registros con producto_id sin match en dim_productos.")

# ----- Resultado -----
if errores == 0:
    logger.info("✅ Validaciones PASADAS (0 errores).")
    print(f"OK - Validaciones PASADAS. Log: {LOG_FILE}")
else:
    _error_and_raise(errores)

if __name__ == "__main__":
    try:
        main()
    except Exception:
        logger.exception("Fallo en validaciones")
        raise

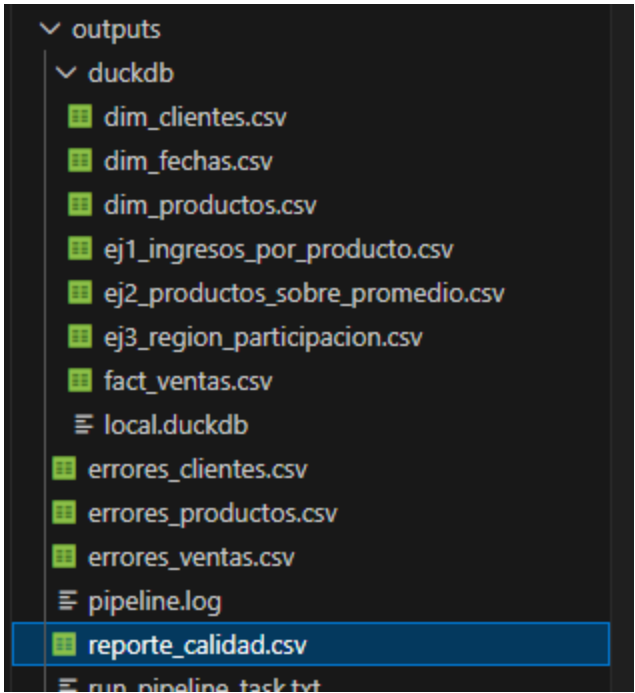
```

```

026-01-01 22:00:37,476 | INFO | == Inicio validaciones (Semana 6) ==
2026-01-01 22:00:37,601 | INFO | \u2705 Validaciones PASADAS (0 errores).
2026-01-01 22:00:58,105 | INFO | == Inicio validaciones (Semana 6) ==
2026-01-01 22:00:58,172 | INFO | \u2705 Validaciones PASADAS (0 errores).

```

tabla,filas_raw,filas_ok,errores
clientes,10000,3862,6138
productos,500,500,0
ventas,200000,30536,169464



```
(venv) PS C:\Users\mlope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador> .\venv\Scripts\python.exe pipeline\validaciones.py
OK - Validaciones PASADAS. Log: C:\Users\mlope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador\outputs\validaciones.log
(venv) PS C:\Users\mlope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador> .\venv\Scripts\python.exe run_pipeline.py
OK - Pipeline completo ejecutado.
Log: C:\Users\mlope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador\outputs\run_pipeline.log
DB : C:\Users\mlope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador\outputs\duckdb\local.duckdb
Out: C:\Users\mlope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador\outputs\duckdb
(venv) PS C:\Users\mlope\OneDrive\Escritorio\MLopezCastro\Data Engineering Bootcamp\ProyectoIntegrador> 
```

